



ZK Theta Language

Developer AI Agent Framework

PROPOSED SPEC · AGENT-NATIVE · POLICY-FIRST

A reference architecture for building trustworthy, observable AI agents.

Manus AI · 2026-08-25

Contents

1	Executive summary	1
2	Why a language layer for agents	1
3	System architecture	1
4	Language anatomy	2
5	Runtime lifecycle	3
6	Trust model	5
7	Developer workflow	5
8	Capability maturity roadmap	5
9	Illustrative design targets	6
10	Multi-agent workflow example	6
11	Cryptography and verification boundary	8
12	Repository reference layout	9
13	Adoption risks and next steps	10
14	Glossary	10
15	References	10

Status note. ZK Theta is presented here as a proposed language and runtime design. The metrics in this document are illustrative design targets for a reference implementation, not independently verified production results.

1 Executive summary

ZK Theta is a proposed developer language and runtime framework for building AI agents as production software systems rather than opaque prompt chains. It gives teams a common way to declare intent, constrain execution, type external tools, preserve evidence lineage, evaluate behavior, and operate agents after deployment.

The framework is designed around one simple premise: an agent should be able to explain what it is trying to do, what it is allowed to do, what evidence it used, which tools it called, and why it stopped. ZK Theta makes those answers part of the program model.

Pillar	Developer outcome
Intent-first syntax	Goals, constraints, and success criteria are explicit before execution.
Typed tool contracts	Every external action has schemas, timeouts, retries, and capability scope.
Policy-native execution	Approvals, permissions, privacy, and budget rules are enforced at runtime.
Evidence-bound memory	Observations and inferences remain separable, scoped, and traceable.
Evaluation loops	Plans, tool use, grounding, latency, cost, and approvals are regression-tested.
Deployment observability	Traces, lineage, audit events, and rollout signals are emitted by default.

2 Why a language layer for agents

Modern agent systems combine language models, APIs, databases, browsers, code execution, and human approvals. When these elements are glued together without a shared contract, teams inherit recurring failure modes: actions that cannot be replayed, tools that accept ambiguous inputs, memories that outlive their intended scope, and successful-looking runs that cannot explain their evidence.

A language layer does not remove model uncertainty. Instead, it moves the most important controls into a place that can be reviewed, tested, linted, and enforced. The result is a boundary between probabilistic reasoning and deterministic execution.

Design principle. Let the model propose. Let the runtime validate. Let policy decide. Let evidence explain. Let telemetry make the whole loop operable.

3 System architecture

The ZK Theta architecture separates the agent runtime from the trust plane. Intent and planning stay flexible, while policy, evidence, tool contracts, observability, and evaluation create deterministic control points around execution.

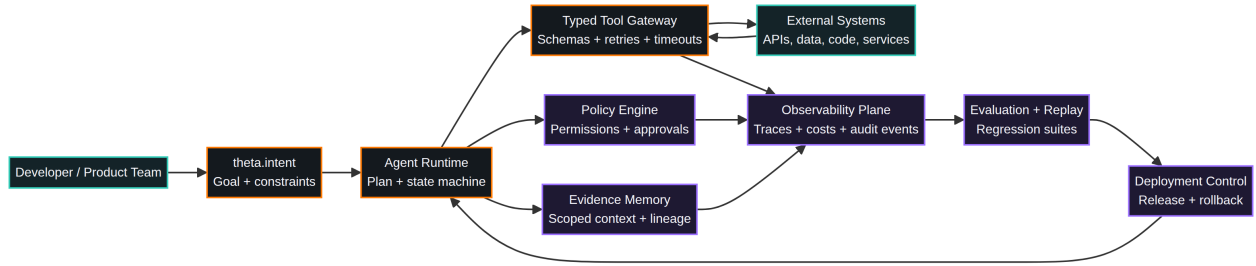


图 1 Proposed ZK Theta system architecture.

The key boundary is the typed tool gateway. A tool call is not merely a function invocation; it is a capability-bearing event with a schema, policy context, timeout, retry behavior, result validation, and audit identity. External systems remain replaceable behind that boundary.

4 Language anatomy

A ZK Theta program declares the agent's role, goal, constraints, tools, plan, and observability requirements in one reviewable unit. The syntax below is intentionally compact: it demonstrates the design direction rather than claiming a finalized compiler grammar.

```

agent ReleasePilot {
  goal "prepare a safe production release"
  constraints {
    environment == "production"
    budget_usd <= 25
    require_approval for deploy
  }

  tools {
    inspect_ci: Tool<BuildReport>
    open_change: Tool<ChangeRequest>
    deploy: Tool<DeploymentReceipt> policy guarded
  }

  plan {
    evidence = inspect_ci.run()
    assert evidence.status == "green"
    change = open_change.create(evidence)
    await approval("release-manager")
    deploy.run(change)
  }

  observe { trace, costs, policy_events, evidence_lineage }
}

```

The language design intentionally makes unsafe behavior visible. The `require_approval` clause is not a comment for a human reviewer; it is a runtime obligation. The `assert` clause turns an evidence condition into a gate. The `observe` clause declares the operational record expected from the run.

5 Runtime lifecycle

Each run follows a lifecycle that can be replayed and inspected. A failed policy gate returns an explanation or requests approval. An invalid tool result triggers recovery or escalation rather than silently contaminating downstream state.

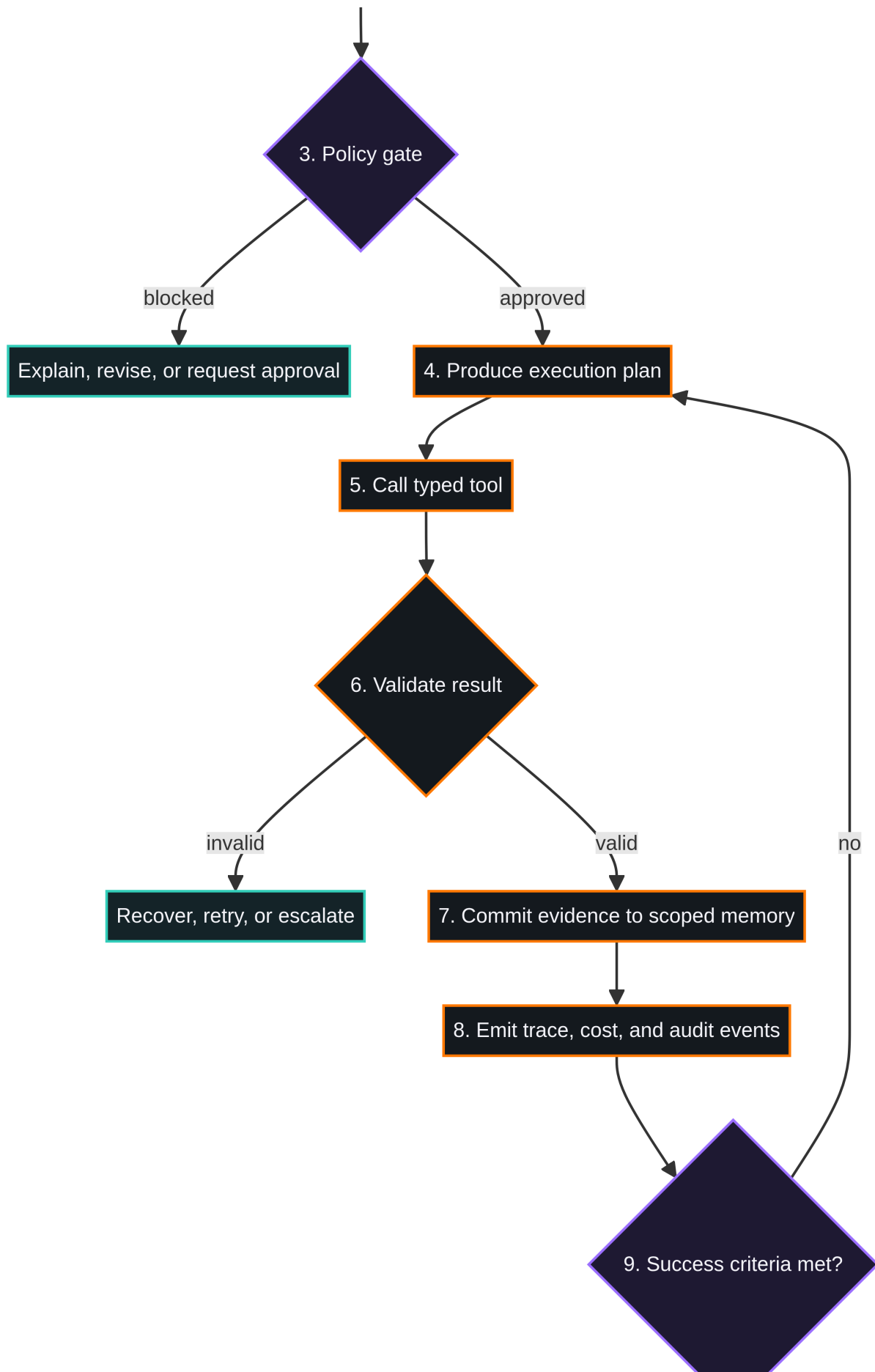


图 2 Proposed runtime lifecycle for a ZK Theta agent run.

6 Trust model

ZK Theta uses four complementary controls. **Policy** defines what may happen. **Evidence** records what was observed and how it entered the run. **Approval** provides an explicit human or service decision at a high-impact boundary. **Auditability** preserves the event trail needed for review, incident response, and replay.

Control	Question answered	Implementation shape
Policy	What is allowed?	Capability scopes, deny rules, budgets, environment gates, approval requirements.
Evidence	What supports the decision?	Source references, timestamps, hashes, confidence, and lineage links.
Approval	Who authorized the risk?	Named approver, decision state, expiry, and change context.
Auditability	What actually happened?	Immutable event IDs, tool receipts, traces, costs, and replay inputs.

7 Developer workflow

The intended workflow is familiar to software teams: design a contract, implement a small capability, run fixtures, inspect a trace, and promote only when the evidence is sufficient. ZK Theta adds agent-specific artifacts to that loop without hiding them behind a platform-specific dashboard.

1. **Declare.** Write the goal, constraints, tools, and success criteria.
2. **Compile.** Resolve schemas, permissions, environment references, and policy obligations.
3. **Simulate.** Run fixtures with mocked tools and adversarial inputs.
4. **Evaluate.** Compare expected evidence, tool behavior, latency, cost, and approvals.
5. **Deploy.** Release with trace collection, rollback controls, and environment-specific policy.
6. **Replay.** Reproduce important runs from their event and evidence records.

8 Capability maturity roadmap

The maturity model below provides a practical adoption sequence. Teams can begin with typed tools and observability, then add policy gates, evidence lineage, and replay as their risk profile grows.

Stage	Capability	Exit signal
1 · Instrument	Trace every run and record tool receipts.	Operators can inspect what happened.

2 · Contract	Type tools, validate results, and standardize errors.	Tool failures are actionable and replayable.
3 · Govern	Add policy gates, approvals, budgets, and scoped memory.	High-impact actions are controlled.
4 · Evaluate	Run regression fixtures and adversarial tests.	Releases are compared against known behavior.
5 · Operate	Add rollout, rollback, cost controls, and SLO-style signals.	Agent behavior is manageable in production.

9 Illustrative design targets

The following chart is a planning aid for a reference implementation. It shows the kinds of measurable outcomes the framework should make possible; it does not report results from an existing production deployment.

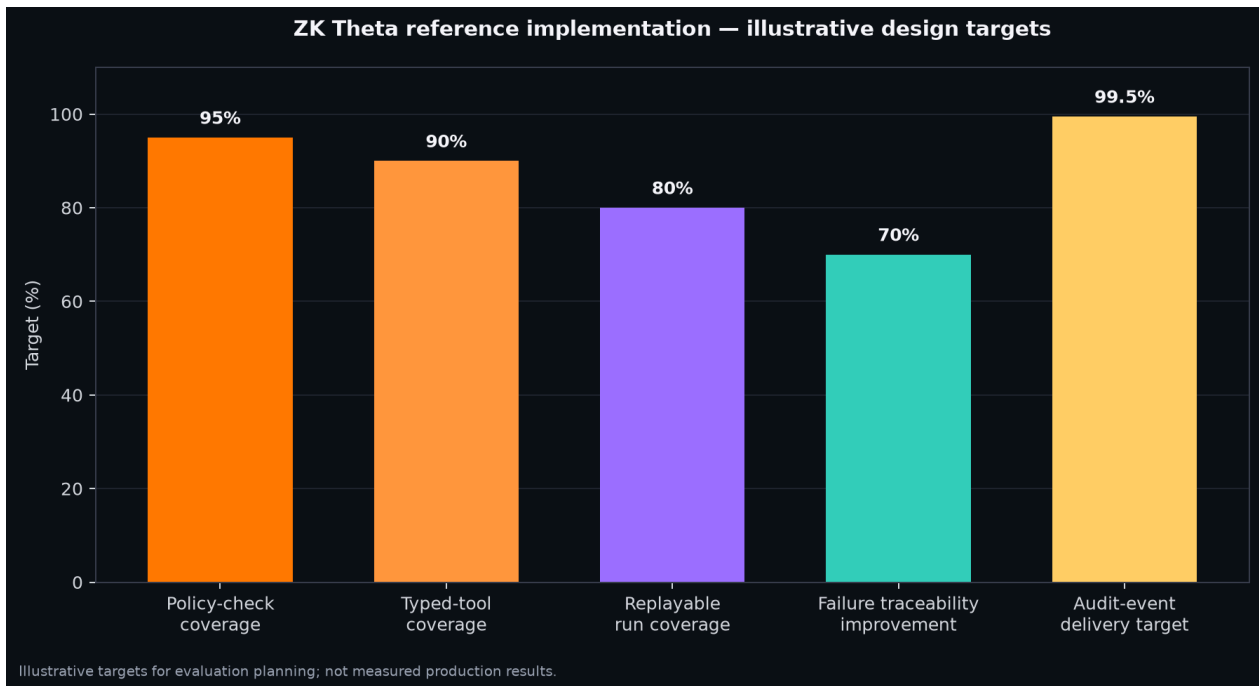


图 3 Illustrative targets for evaluating a ZK Theta reference implementation.

10 Multi-agent workflow example

ZK Theta is designed to make handoffs between specialized agents explicit. The example below defines a Planner, Researcher, Verifier, and Promoter. Each handoff carries structured state, and the final promotion is blocked until evidence assertions and an approval gate succeed.

```
workflow SafeRelease {  
  input request: ReleaseRequest  
  policy {
```



```
environment == "staging"
max_budget_usd <= 5
require_approval for promote
}

agent Planner {
  goal "turn a release request into an ordered execution plan"
  tools { read_request: Tool<ReleaseRequest> }
  plan {
    request = read_request.run()
    return Plan { checks: ["ci", "security", "change-window"] }
  }
}

agent Researcher {
  goal "collect independent evidence for every release check"
  tools { ci_report: Tool<CIReport>; security_report: Tool<SecurityReport>;
change_window: Tool<ChangeWindow> }
  plan {
    ci = ci_report.run(request.repository)
    security = security_report.run(request.commit)
    window = change_window.run(request.service)
    assert ci.status == "green"
    assert security.status == "pass"
    assert window.open == true
    return EvidenceBundle { ci, security, window }
  }
}

agent Verifier {
  goal "validate evidence and produce a promotion receipt"
  plan {
    assert evidence.count == 3
    return VerificationReceipt { decision: "eligible" }
  }
}

agent Promoter {
  goal "promote only after a valid verification receipt and approval"
  tools { promote: Tool<PromotionReceipt> policy guarded }
  plan {
    assert verification.decision == "eligible"
```

```

    await approval("release-manager")
    return promote.run(request.service, request.commit)
  }
}

pipeline {
  request = Planner.run(input)
  evidence = Researcher.run(request)
  verification = Verifier.run(evidence)
  result = Promoter.run(request, evidence, verification)
}
}

```

The repository includes a standard-library Python reference runner for this workflow. It demonstrates execution, evidence assertions, explicit approval, and hash-linked audit receipts, but it is not a compiler for the proposed language and does not generate zero-knowledge proofs.

11 Cryptography and verification boundary

ZK Theta does not currently implement a zero-knowledge proof system. The honest design answer is therefore that no cryptographic primitive in the current repository guarantees zero-knowledge verification. A future implementation would need a formally specified transition relation, a sound and zero-knowledge proof system, canonical serialization, secure transcript binding, and independent review. The proposed modular suite is as follows:

Component	Proposed primitive	Role
Hashing	SHA-256 or SHAKE256	Domain-separated policy, transcript, state, and event digests.
State commitments	Merkle trees over hashed events	Commit to ordered evidence logs and prove inclusion.
Transition proof	Transparent STARK-style IOP/FRI construction	Prove a private witness satisfies the public state-transition relation without a trusted setup.
Non-interactive proof	Fiat-Shamir transcript	Derive verifier challenges from canonical commitments and public inputs.
Identity and approvals	ML-DSA; Ed25519 only for a non-PQ prototype	Authenticate tool manifests, approvals, and deployment receipts.

Confidential transport	ML-KEM	Establish keys for encrypted evidence or telemetry channels.
------------------------	--------	--

A hash, signature, or key-encapsulation mechanism is not itself a zero-knowledge proof. Zero knowledge depends on the relation and the proof system’s formal security properties. The proposed STARK-style route is a design choice intended to avoid a trusted setup; a production team could select another proof family, but it would need to document its assumptions, soundness error, privacy property, setup model, field, recursion strategy, and implementation status.

The verification flow is: compile the workflow into a relation and policy digest; serialize public inputs and private witness data canonically; commit to private state and ordered events; generate a non-interactive proof; authenticate the receipt and approval metadata; and then verify the proof, commitments, signatures, policy digest, and chain linkage. Private witness data must remain outside the public statement for privacy to hold.

Security qualification. The IRTF Fiat–Shamir document referenced here is an active Internet-Draft, not a final standard. NIST’s ML-KEM, ML-DSA, and SLH-DSA are post-quantum standards for key establishment and signatures; they do not by themselves prove an agent transition in zero knowledge.

12 Repository reference layout

A reference repository should keep the language proposal, executable examples, diagrams, and generated artifacts close together while preserving a clear path toward future compiler and runtime work.

```
zk-theta-framework/
├─ README.md
├─ LICENSE
├─ CONTRIBUTING.md
├─ docs/
│   └─ framework.md
│   └─ zk-theta-framework.pdf
├─ diagrams/
│   └─ architecture.mmd
│   └─ runtime-lifecycle.mmd
├─ assets/
│   └─ zk-theta-icon.png
│   └─ architecture.png
│   └─ runtime-lifecycle.png
├─ data/
│   └─ create_target_chart.py
└─ src/
    └─ example.theta
```

13 Adoption risks and next steps

The largest risk is confusing a compelling syntax with a complete safety model. A credible implementation must define schema validation, policy precedence, secret handling, memory retention, approval expiry, and failure semantics before it optimizes developer ergonomics. Another risk is over-instrumentation: telemetry should be useful, privacy-aware, and governed by explicit retention rules.

The next practical step is a small reference runtime that can compile one agent definition, execute mocked typed tools, enforce one approval gate, and emit a replayable trace. That slice is sufficient to validate the central thesis before a larger language surface is designed.

14 Glossary

Term	Definition
Agent	A software process that combines model reasoning with tools, state, and execution policies.
Evidence	An observed input or result with source, timestamp, and lineage metadata.
Policy gate	A deterministic runtime decision that permits, blocks, or escalates an action.
Tool contract	The schema and execution rules governing an external capability.
Replay	Reconstructing a prior run from recorded inputs, decisions, tool receipts, and evidence.

15 References

This proposal is intentionally self-contained. The following standards and public references provide relevant background for security controls, software supply-chain integrity, and responsible AI risk management.

1. National Institute of Standards and Technology, **AI Risk Management Framework (AI RMF 1.0)**, 2023. nist.gov/itl/ai-risk-management-framework.
2. National Institute of Standards and Technology, **Secure Software Development Framework (SSDF)**, SP 800-218. csrc.nist.gov/Projects/ssdf.
3. OpenTelemetry Documentation, **Observability framework documentation**. opentelemetry.io/docs.
4. NIST, **Post-Quantum Cryptography Standards**. nist.gov.
5. IRTF, **Fiat-Shamir Transformation**, draft-irtf-cfrg-fiat-shamir-03. datatracker.ietf.org.